Skis
CS 3435 Final Project

Leif Nyland

This project involves collecting data on skis in an attempt to predict the price of skis, as well as reveal the exact influence of various qualities of skis on the overall price. The ski information comes from evo.com, a popular online retailer. The data is analyzed to reveal any potential noteworthy trends, and the Python package SciKitLearn provides a framework to make illuminating models used for prediction.

Data

The data for this project are collected from Evo.com using scrapy, a web scraper written in python. Evo.com contains 840 ski item pages on its website at the time of this report. The item name, item number, price, company of manufacture, key materials used in manufacture, target ability level, target terrain, turning radius, and url are collected and stored in a csv file for analysis.

Visualization

Most of the visualizations demonstrate how the properties of a ski may have an impact on the price using bar graphs. If the properties of a ski affect the price, then it may be possible to predict the price of a ski based on one or more properties.

Models/Predictions

Most of the models use linear regression to predict a price of a ski based on one or more factors. Analyzing the coefficients for the linear regression models leads to interesting conclusions about each category because the coefficient reflects the amount that each category affects the price of a ski, on average. Using the fit_intercept hyperparameter is key to getting consistent, small coefficients because it allows the intercept to represent a better average ski price. Using Ridge linear regression is also beneficial for producing small coefficients because it minimizes the square of the coefficients.

The models used to predict categories of skis proved less effective, but still enlightening. Using GaussianNB to predict manufacturing companies by grouping was unsuccessful with the models used, with a 0.152 accuracy score at best. Trying to see a visual representation of the groupings is the next step, to see if it appears possible. Isomap with n_components=2 reduces the dimensions down to 2, which allows the data to be graphed on a 2D plane. Each point is colored to represent its company.

Results

The linear regression appears to be moderately successful. Most of the models experienced a root mean squared error of roughly 330, with the best model sitting right under

300. Although a predicted price with an error of $300 is minimally useful, the main benefits of this model come from the coefficients. The coefficients reveal the estimated effects of individual features on the price of a ski.

The 2D projection of the data to look for company groupings was not successful in the sense that each company had its own bubble, but some definite trends emerge. The shape of the overall graph is very intriguing, and more research to reveal the meaning of the shape is required. Also noteworthy, some color groupings do emerge. For example, the long tails of the scatter plot seem to be primarily limited to the company represented in blue. More research is required.

Conclusions

Interestingly, of the linear regression models based on only one major category, the one based on the target terrain of the ski is the most effective at predicting the ski price with a root mean squared error of roughly 303. Based on the coefficients, big mountain and alpine touring skis seem to be more expensive in general, while park & pipe skis are significantly cheaper than the rest. Similar information can be revealed by analyzing the coefficients for the rest of the models.

How to run the code

1. Make sure the working directory is inside `/final_project3_cs3435/final_project`
2. Run the command `$scrapy crawl ski_info`. This will take roughly 2 hours
3. This should produce a csv file called ski_info.csv with all of the ski data. My file is inside `/final_project3_cs3435/final_project/`
4. Open the python file ski_graphs.py. If it is run as is, all graphs and models will be run.
5. Near the very bottom of the file, there is a `main()` function with only 2 lines of code: `graphs()` and `models()`
6. Commenting out one of these will completely stop all of that category from running. `graphs()` produces graphs and visualizations, and `models()` produces the models trained on the data.
7. Additionally, you can scroll up to the definition of either `graphs()` or `models()`. Near the bottom of each method, all possible graphs or models are listed, and any line may be deleted or commented out to stop that specific piece from running.
8. `graphs()` also contains a commented out line that writes the one-hot encoded vectors to a cleaned csv file.